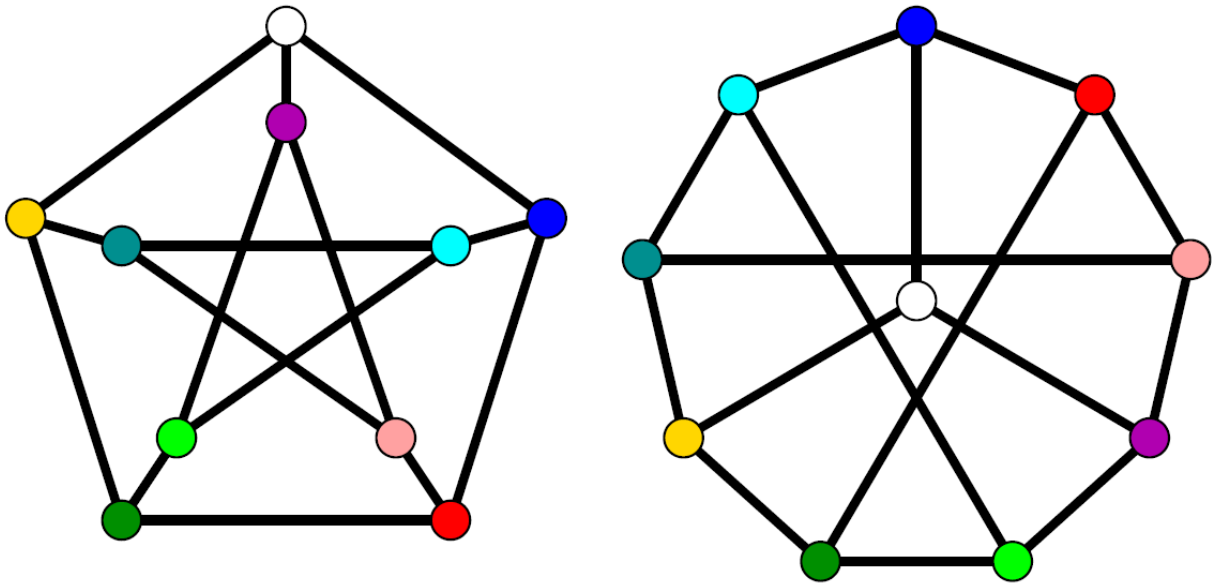
# Implementation Project: Graph Isomorphism



## Kick-off

Ruben Hoeksma

# What this course is about

Learn to:

- translate mathematical concepts into algorithms and implement them into a (Python) program;
- choose the right data structure(s) for the practical implementation of algorithms;

and

- work in a group on an implementation project that has multiple modular parts;
- create and document computational results from (different versions of) the implementation of an algorithm.

# Why do we care about mathematical concepts?

How would you implement a function that outputs $S(n) = 1 + 2 + \cdots + n$?

```python
def summation(n):
    total = 0
    for i in range(n+1):
        total = total + i
    return total
```

- Straightforward
- Running time linear in $n$:
  $n = 10^7$ takes $\sim 3.5$ s
  $n = 10^8$ takes $\sim 35$ s
  $n = 10^{80}$ takes $\sim 35 \times 10^{80}$ s

Compare: Age of the universe $< 10^{18}$ s!

```python
def summation(n):
    total = sum(range(n+1))
    return total
```

- Uses knowledge of Python
- Makes use of build-in functions
- Running time (still) linear in $n$:
  $n = 10^7$ takes $\sim 0.5$ s
  $n = 10^8$ takes $\sim 5$ s
  $n = 10^{80}$ takes $\sim 5 \times 10^{80}$ s

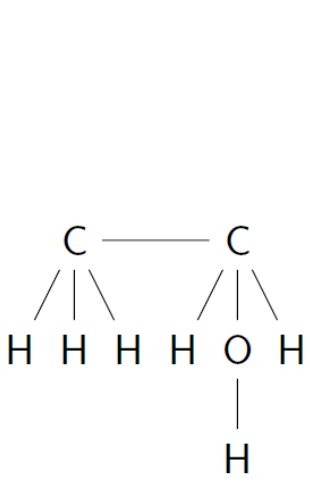
```python
def summation(n):
    total = n*(n+1)//2
    return total
```

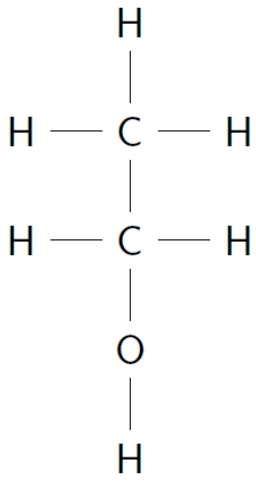
- Uses knowledge of math
- Running time constant in $n$:
  $n = 10^7$ takes $< 0.00001$ s
  $n = 10^8$ takes $< 0.00001$ s
  $n = 10^{80}$ takes $< 0.00001$ s

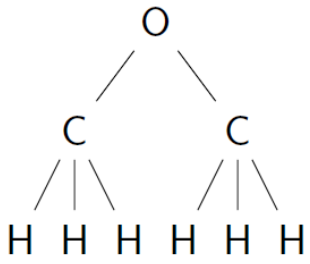
# About the subject of graph isomorphism

Biochemistry:

- Database of molecules, and a newly found molecule.
- Is it *really* new, or already in the database?



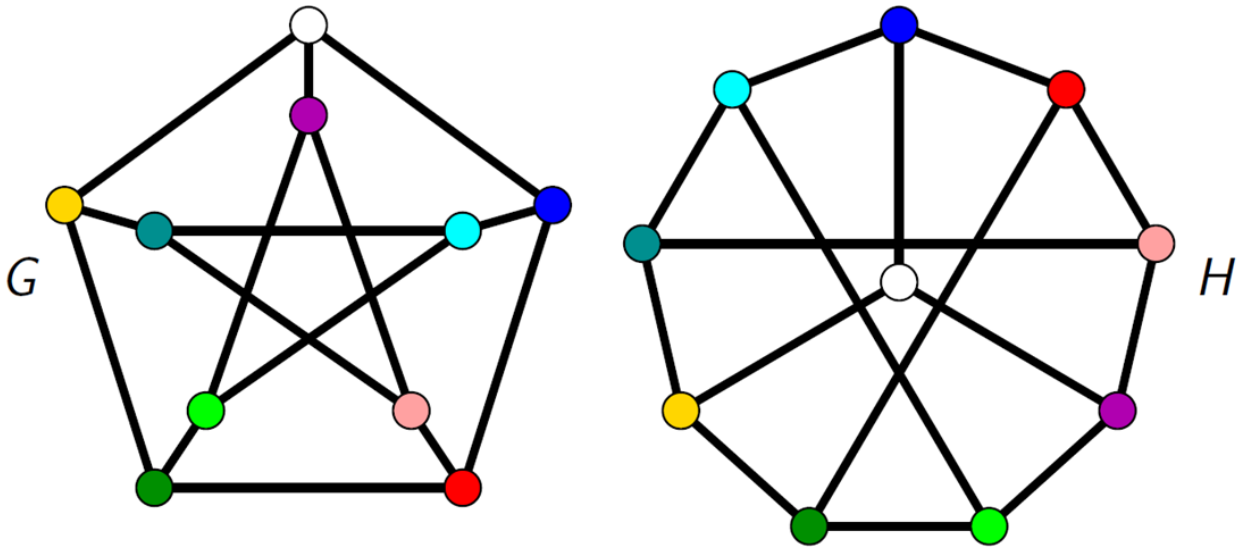




Question: How do you even represent a molecule?

- Several options, common are drawings or lists of atoms and connections.

Solved by Graph Isomorphism

# Symmetry & Similarity: Graph Isomorphism




- An *isomorphism* between two simple graphs $G$ and $H$ is a bijection $f: V(G) \to V(H)$ such that for all $u, v \in V(G)$:
  - $\{u, v\} \in E(G) \Leftrightarrow \{f(u), f(v)\} \in E(H)$ .
- Two graphs $G$ and $H$ are *isomorphic* if there exists an isomorphism between them.
- The *Graph Isomorphism (GI) Problem*: given two graphs $G$ and $H$, are they isomorphic (or not)?

# Other Applications

Variants of GI play a role in many settings:

- electronic circuits,
- graph transformations,
- image recognition,
- fingerprint comparisons,
- solving mathematical models (MIP, CP,...),
- recognizing copied code in computer programs, etc...

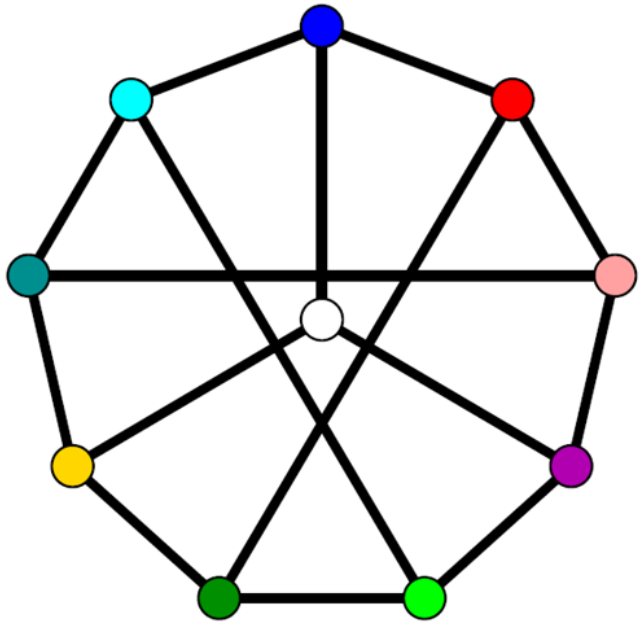
Solving the GI problem efficiently for all graphs (in polynomial time) would be a prize-winning achievement!

# Computational Complexity

How hard is it to solve instances of the GI problem?

- Trying all maps from $V(G)$ to $V(H)$ takes too long ($|V(G)|!$ is big).
- After more than 40 years of research, no efficient (polynomial time) algorithm for GI is known…
- Strong evidence that the problem is not *NP-hard*. ("recent" paper by Lazlo Babai and subsequent discussions…)
- Some algorithmic techniques are known; work very well in practice.

# Symmetry: Counting Automorphisms (#Aut)



- An *automorphism* of a simple graphs $G$ is a bijection $f: V(G) \to V(G)$ such that for all $u, v \in V(G)$:
  - $\{u, v\} \in E(G) \Leftrightarrow \{f(u), f(v)\} \in E(G)$ .
- Any graph has at least one automorphism: $f(v) = v$.
- The *Counting Automorphisms (#Aut) Problem*: given graph $G$, how many automorphism does $G$ have?

# This project

- Write python code that can solve the GI problem and #Aut
- *free how you want to solve this problem*, as long as you can <u>correctly solve</u> the problem for 'reasonably sized instances'
- Five lectures: techniques that are *useful*
- First lecture (next Tuesday): *A custom graph class for Python*
- Second lecture (next week Friday!): *Colour Refinement (Individual Assignment)*